Secure Teleoperation of a Robotic Arm using Computer Vision



Presented by:
Neo Vorsatz

Prepared for:
A/Prof Daniel Ramotsoela
Dept. of Electrical Engineering
University of Cape Town

Submitted to the Department of Electrical Engineering at the University of Cape Town in partial fulfilment of the academic requirements for a Bachelor of Science degree in Mechatronics

October 28, 2026

Declaration

1. I know that plagiarism is wrong. Plagiarism is to use another's work and pretend that it is one's own.
2. I have used the IEEE convention for citation and referencing. Each contribution to, and quotation in, this report from the work(s) of other people has been attributed, and has been cited and referenced. Any section taken from an internet source has been referenced to that source.
3. This report is my own work, and is in my own words (except where I have attributed it to others).
4. **I have not paid a third party to complete my work on my behalf. My use of artificial intelligence software has been limit to [see Appendix E] (specify precisely how you used AI to assist with this assignment).**
5. I have not allowed and will not allow anyone to copy my work with the intention of passing it off as his or her own work.
6. I acknowledge that copying someone else's assignment or essay, or part of it, is wrong, and declare that this is my own work.

Signature:. . . . . .
N. Vorsatz

Date:. . . . . 28/10/2026 . . . . . .

Word count: 8418
(excl. headings, tables, figures, equations and footers)

Terms of Reference

The student is expected to deliver a cohesive hardware and software architecture that successfully enables real-time, non-contact teleoperation of a physical manipulator. Final deliverables must encompass a formally documented spatial translation framework, the underlying signal processing methodology, and a rigorous benchmarking analysis that quantifies the system's performance, safety, and operational boundaries under dynamic input conditions. The student must also deliver an extensive attack surface analysis of the system and a comprehensive literature review and engineering trade-off analysis justifying the chosen approach.

Acknowledgments

I send my thanks to my supervisor, A/Prof Ramotsoela, for keeping me on-track with completing this project's goals, as opposed to drifting off on a tangent.

I send my thanks to Shaun Beautement for bringing to my attention the Hairy Ball Theorem, and its implications on a static kinematic mapping.

I send my thanks to Lusango Mbongo for developing the initial designs of the 3D printed mount, which was intended to hold the Braccio and attached modules. The final design is incomplete as of writing this report, but will be added to the GitHub repository once completed.

I send my thanks to an anonymous person for recommending that I look into virtual private networks.

Lastly, I send my thanks to my friends and family, who all indirectly supported this project.

Abstract

Teleoperation systems are used to allow humans to control robotic systems over large distances. An ideal user experience involves non-intrusive sensing and intuitive spatial mapping. Few implementations of teleoperation discuss the effects of applying cybersecurity principles to the resource constrained hardware inherent to some robotic systems. This report discusses the implementation of an intuitive and non-invasive teleoperation system, using computer vision, while keeping network security and constraints in consideration. The system uses a pre-trained computer vision pipeline, along with an inverse kinematics algorithm to determine the actuation values for a Tinkerkit Braccio. Packets sent from the operator-side to the Braccio-side are managed by the DTLS protocol, using pre-shared keys. Due to monocular depth ambiguity, the motion of the Braccio was constrained to 2 dimensions. Through experiments, it was found that the smoothing algorithm used to reduce aggressive movements contributed most (88.3%) to the system's total latency. The latency due to encryption and decryption were otherwise insignificant, and this is more apparent over slower networks, such as due to large geographical separation. The implementation code, report source code, raw data and videos can all be found on the project's GitHub.

Contents

1 Introduction **1**

1.1 Background to the project 1

1.2 Objectives of this project 1

1.3 Scope and Limitations 1

1.4 Plan of development 2

2 Literature Review **3**

2.1 Introduction 3

2.2 Overview 3

2.3 Kinematic Mapping 5

2.4 Inverse Kinematics 6

2.5 Computer Vision 8

2.6 Network 9

3 Computer Vision **10**

3.1 Tools 10

3.2 Calibration 11

3.3 Target Generation 11

3.4 Testing 12

4 Inverse Kinematics **13**

4.1 Requirements 13

4.2 Algorithm 13

4.3 Implementation for the Braccio 16

4.4 Testing 17

4.4.1 Acceptance Test Procedures 17

4.4.2 Benchmarking 17

5 Network **18**

5.1 Internet Protocol 18

5.1.1 Protocol 18

5.1.2 Payload Format 19

5.1.3 Tools 19

5.2 Communication with Arduino Uno 20

5.2.1 Protocol 20

5.2.2 Payload Format 20

5.3 Testing 21

5.3.1 Penetration Testing 21

5.3.2 Benchmarking 21

6 Actuation **22**

6.1 Braccio Control Library 22

6.2 Main Loop 23

6.3 Testing 23

7 Results **24**

7.1 Latency 24

7.2 Inverse Kinematics Solver 25

8 Discussion **27**

8.1 Latency 27

8.2 Inverse Kinematics Solver 28

8.3 Cybersecurity 28

9 Conclusions **29**

10 Recommendations **30**

A GA Appendix **35**

B Ethics Approval Letter **36**

C Inverse Kinematics Pseudocode **37**

D Example IK Acceptance Test Scenarios **41**

E AI Usage **43**

E.1 Inline Code Suggestions 43

E.2 Google Gemini's Advice on LaTeX syntax 44

Chapter 1

Introduction

1.1 Background to the project

Teleoperation is the control of systems (often robotic) over large distance. This is done because it's often useful to have a human in the loop as opposed to fully automated systems. Teleoperation poses many modern challenges, such as keeping the network secure and responsive on resource constrained hardware. Non intrusive and intuitive methods of reading user input, such as computer vision, are also preferred because they introduce less fatigue and strain on the operator.

1.2 Objectives of this project

This project aims to develop a teleoperation system using the Tinkerkit Braccio for Arduino and an ESP32, by implementing non-intrusive computer vision to read user input, and keeping the network security in mind.

This is done to demonstrate and investigate the use of computer vision and intuitive control schemes in teleoperation systems. This project also discusses the overall cybersecurity of this system, and investigates the effects of implementing cybersecurity principles on the network's performance, given resource constrained hardware.

1.3 Scope and Limitations

This project assumes an ideal operator environment, where the robustness of the computer vision is not a concern.
This project does not implement a feedback system for the operator, and hence assumes the operator already has some way to see the slave's and its environment.
This project also focuses on the implementation of cybersecurity for the internet traffic, as other security such as for the hardware or encryption keys would not significantly affect the performance of the teleoperation.
This project does not investigate the overall accuracy of the teleoperation.

1.4 Plan of development

In chapter 2, is a review of similar projects and studies, and the technologies surrounding the subsystems for this project.

From chapter 3 to chapter 6 are the explanations of the design and implementation of each of this project's subsystems:

- chapter 3 is dedicated to the computer vision subsystem, explaining how user input is taken and interpreted.
- chapter 4 goes through the implementation of the inverse kinematics subsystems, which maps the desired end-effector target to a set of actuation/control values for the Braccio.
- chapter 5 discusses some risks the network faces, and how those risks were mitigated in the networking system.
- chapter 6 explains how the Braccio was actuated to achieve the desired end-effector target.

The results are shown and discussed in chapter 7 and chapter 8. The outputs of this project are summarised in chapter 9, and recommendations for future work are made in chapter 10.

Chapter 2

Literature Review

2.1 Introduction

The application areas for robotics continues to expand, in both industrial and household contexts. Fully autonomous robots still pose risks in many contexts such as surgery, hence why it's often preferred to have a human in the control loop. Teleoperation refers to the control of machines, vehicles or robots over a distance, allowing human discretion and intuition to be incorporated into these contexts.

Operators may not always be able to receive training on how to operate robotic systems, as formal training can be very time consuming for individuals and for companies. Intuitive forms of teleoperation aim to solve this, by reading an operator's natural movements or gestures and mapping these to corresponding commands (such as setpoints) for the robot systems. These natural movements can be observed with computer vision, which is far more accessible than specialised sensory hardware. Regardless of the data capture method, intuitive teleoperation introduces the challenge of mapping human kinematics to corresponding kinematics for robotic limbs, which often aren't anthropomorphic.

Another risk that arises with teleoperation is weak network security. Malicious actors may wish to hijack robotic systems, steal configuration data, or even hold robotics systems under ransom. Teleoperation systems have realtime performance requirements, hence why implementing network security without introducing significant latency is an ongoing discussion.

2.2 Overview

Receiving input from the operator has been done either via computer vision or a sensor suite attached to the operator's arm and/or hand. In a demonstration done by Mongardi *et al.* [1], success was found in teleoperating the Braccio arm using an armband that read muscle flexes to determine the operator's pose. An artificial neural network was used to convert the flexing data into 8 different poses. In the works of Reyes *et al.* [2], one of the modes of operation used to teleoperate the Braccio arm uses computer vision to detect hand gestures. This offers a discrete set of operator poses, each corresponding to a pre-defined pose for the Braccio arm. They designed the poses for the Braccio arm such that the poses could be sequenced to complete a teleoperation task.

Drăgoi *et al.* [3] teleoperate a robotic arm with similar kinematics to the Braccio arm. They use the OpenCV software to extract the video feed from a built-in camera on a laptop, and Google's MediaPipe Hands software to identify the pose of the operator's hand. MediaPipe Hands provides monocular hand tracking and pose estimation [4]; it uses the feed from a single camera to provide the coordinates (3-dimensional space) of several landmarks on a human hand.
The team addresses kinematic mapping by first having a calibration step (to improve repeatability among different operators), and then using descriptors (such as "hand pitch" and "horizontal displacement") to control individual joints of the robotic arm. Alternatively using end-effector mapping with inverse kinematics was briefly discussed, but this was opted out as it would reduce stability and introduce more modelling overhead. The paper suggests end-effector mapping with IK as a natural next step.
They address stability of the teleoperation system by; (1) low-pass filtering the setpoints of each joint, using a moving-average filter over 3 frames, (2) limiting the framerate of setpoints to 1 per 0.03s, thereby reducing network traffic and high-frequency oscillations in the setpoints, and (3) applying a deadband of $\pm 2^\circ$ to the joint angles, to reduce jitter.
Drăgoi *et al.* chose to send commands to the robotic arm over a local HTTP network, where the robot hosts the server using the HTTPServer and BaseHTTPRequestHandler software. They do not address the cybersecurity of the network.
With these decisions, the project found some success, having to trade off accuracy for accessibility. The primary causes of error identified were: (1) inconsistent camera placement and hand visibility, (2) monocular depth ambiguity, and (3) the operator's familiarity with teleoperation. The paper recommends multi-view capture to improve independence of camera placements, and eliminate monocular depth ambiguity. Another recommendation of note is to use task-aware smoothing, giving gross movements less smoothing, and finer movements more.

Chen *et al.* [5] developed a software platform, which they call TeleopLab, to bring intuitive teleoperation into educational contexts. This allowed students at home to perform practical activities, such as laboratory experiments.
They chose for smartphones to be used by the operators (students) because of how common they are. The smartphone's relative position to an origin is determined using camera frames and motion sensing. This displacement is sent to the host computer at the lab using the public and forwarded port of the computer. The network and the commands were set up using the TeleMoMa framework [6].
When controlling the robotic arm, they map the cartesian coordinates of operator's pose to the end-effector of the robotic arm. The inverse kinematics was handled using the TRAC-IK solver, which was plugged into the ROS. TRAC-IK runs two inverse kinematics methods simultaneously, and returns whichever converges first, typically within 1 millisecond [7]. The first method is an extension of the Newton-Raphson method that mitigates local minima, and the second is a sequential quadratic programming optimisation.
The overall project was successful in its goal, receiving positive feedback from the students. The learning/improvement rate was notably slower than with other teleoperation systems [8][9][10], but this may be attributed to the fact that the novice operators were younger in age.

The security of networks is rarely discussed in conjunction with other teleoperation subsystems. In articles that do discuss security in teleoperation, the security has been the focus of the paper with little attention on other subsystems.

2.3 Kinematic Mapping

Mapping the kinematics of one device (the master limb) to the kinematics of another device (the slave limb) has generally been achieved by identifying positional characteristics of the master limb, and using that to calculate the corresponding positional characteristics of the slave limb, using a linear transformation.

Joint mapping is where the positions of individual joints from the master limb is mapped to the positions of individual joints on the slave limb. This method works well when the master and slave share very similar kinematics [11]. The mapping Drăgoi *et al.* [3] use is very similar to joint mapping, in which they map the master's positional descriptors to each of the slave's joints.

Another method is **end-effector mapping**, sometimes called "cartesian mapping" (and very similarly "fingertip mapping"). In this mapping method, the coordinates of the master's end-effector are mapped to coordinates for the slave [11]. The positions of joints in the slave are then determined via inverse kinematics.
Wang *et al.* [8] evaluate end-effector mapping, joint mapping, and a unique "steering mapping" (which can be described as a variant of joint mapping) in terms of human-centric measures, such as cognitive load. Compared to the other two, they found that end-effector mapping gave significantly less cognitive load and resulted in less jerking (hesitations and sudden corrections) from the operator. End-effector mapping is also seen implemented in TeleopLab [5].
A variant of end-effector mapping is applied in the mixed-reality project of Su *et al.* [12], in which the displacement required to achieve a desired setpoint is used to determine velocities for each joint as opposed to just their positions.

Pose mapping is where the intention of the master's pose is used to map to a slave-pose with similar intention. This is generally implemented with a discrete set of mapped poses, and interpolating between these poses [13]. Machine learning computer vision is a popular method of identifying the pose of the master limb. Pose mapping has unpredictable non-linear behaviour for poses that were interpolated [11][9].

Meeker *et al.* [9][10] propose **subspace mapping**, which uses an intermediate subspace to map between dissimilar kinematics. The intermediate subspace has basis vectors corresponding to the minimum number of parameters required to describe a pose, such as the "openness", "spread" and "curl" of a human hand. In their later work [10], they describe a methodology for generating the transformation matrices (and other mapping constants) through a series of experiments as opposed to by intuition.
This mapping model was compared to the end-effector and joint mapping models, with respect to how quickly novice operators were able to complete tasks, and how few tries were needed. They found that subspace mapping out-performed end-effector and joint mapping in these metrics. Joint mapping followed closely, in terms of performance, and this was attributed to the fact that joint mapping offered individual control of each robotic joint, which is not necessarily possible with the proposed subspace mapping.
Subspace mapping performs well in these tasks where spatial positioning is not the challenge. Meeker *et al.* intentionally used a different cartesian mapping method to position the end-effector (hand), although this method wasn't discussed in any depth. Using a different mapping method was possibly due to the limited expressiveness that comes with using a low-dimensional intermediate subspace.

The Braccio arm has significantly dissimilar kinematics to that of a human forearm, making joint-mapping less preferable although still plausible, with the variation used by Drăgoi *et al.* [3]. The Braccio also has a relatively simple end-effector; a two-finger gripper (or "claw"). This excludes the primary advantage that subspace mapping has over other mapping methods. Between end-effector mapping and pose mapping, end-effector mapping has been applied significantly more and consistently shown to be effective, making it the preferable option.

2.4 Inverse Kinematics

Given that end-effector (EE) mapping is the desired way to transform between the kinematics of a human forearm with that of the Braccio, a suitable inverse kinematics (IK) algorithm is required. Among IK solvers, there are analytical and numerical methods.

Analytical methods commonly use geometry, like the rule of cosines, to solve for joint angles. These methods are typically the fastest and most accurate [14][15][16] but are limited, because they assume exactly one closed-form solution exists, or a small finite number of solutions [17], among which one is implicitly selected.
When there are more degrees of freedom than constraints, there are infinitely many solutions. Assuming segment lengths are fixed, Figure 2.1 (a) shows a desired EE position in 2 dimensional space (2 constraints), with the only IK solution. We ignore the mirrored solution, because it is usually implicitly discarded when formulating the close-form solution. In diagram (a), analytical methods work. Diagrams (b) and (c) show two of the infinite possible solutions for the same desired EE position, but with 3 degrees of freedom.
Another challenge for analytical solutions is scenarios with no solutions. Such scenarios arise when the desired EE position is outside of the reachable workspace, either due to limited reach or motion constraints on the degrees of freedom.

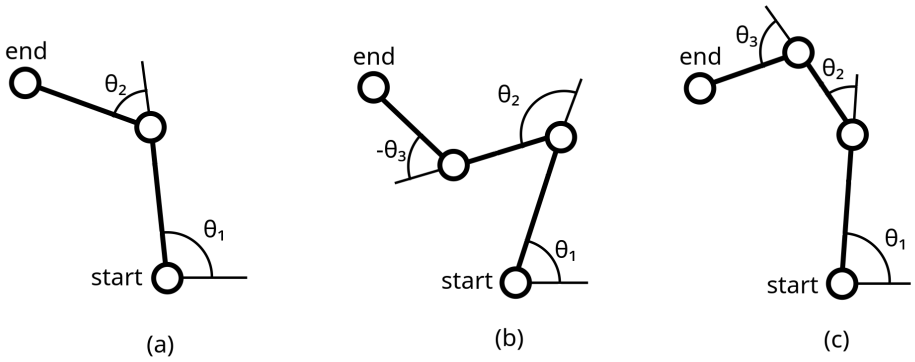


Figure 2.1: Diagram showing IK solutions for different degrees of freedom

Numerical methods typically use an initial guess, then an iterative process to improve that guess until the theoretical EE position is within acceptable error of the desired EE position. These methods handle the cases with infinite or no solutions, but are more computationally expensive.

The most popular ways to perform numerical IK are **Jacobian-based** methodologies [14][15]. By using a system of equations that describe the forward kinematics, the Jacobian for the EE position with respect to the actuations is calculated, giving the relationship shown in Equation 2.1. The Moore-Penrose pseudo-inverse of this Jacobian is used in a Newton-Raphson iterative process, where successive best guesses are generated [14], shown in Equation 2.2.

$$\frac{d\vec{x}}{dt} = J(\vec{q}) \cdot \frac{d\vec{q}}{dt} \tag{2.1}$$

$$\vec{q}_{i+1} = \vec{q}_i + J(\vec{q}_i)^{\dagger} \cdot (\vec{x}_{target} - \vec{x}(\vec{q}_i)) \tag{2.2}$$

Newton-Raphson struggles with singularities, which occur when no solution exists, or the initial guess leads to a local maximum/minimum that does not have a nearby solution. To counter this, a Damped Least Squares procedure can be implemented, which replaces the pseudo-inverse [14], as shown in Equation 2.3. b is a small positive damping factor, and I is the identity matrix.
To meet requirements other than EE position, these Jacobian methods add "secondary tasks", which do not affect the position of the EE, and only the intermediate joints. The **gradient projection method** (GPM) [18] establishes an objective function $H(\vec{q})$ that we want to minimise, and the gradient of this objective function is projected into the null space (where it doesn't affect the EE). This contribution is added to the Newton-Raphson equation, as shown in Equation 2.4. k is a positive scalar that increases the aggressiveness of the objective function. Secondary objectives can include: avoiding joint-limits, or avoiding obstacles by modelling the physical space as a potential field.

$$J^{\dagger}_{DLS} = J^T (J \cdot J^T + b^2 \cdot I)^{-1} \tag{2.3}$$

$$\vec{q}_{i+1} = \vec{q}_i + J(\vec{q}_i)^{\dagger} \cdot (\vec{x}_{target} - \vec{x}(\vec{q}_i)) + k \cdot (J(\vec{q}_i)^{\dagger} \cdot J(\vec{q}_i) - I) \cdot \nabla H(\vec{q}_i) \tag{2.4}$$

The mixed-reality project [12] makes use of a velocity-based variant of the Jacobian method. Instead of computing incremental changes and iterating through the Newton-Raphson method, they used the result from applying the pseudo-inverse as a velocity for the joints.

Forward And Backward Reaching Inverse Kinematics (FABRIK) is a popular heuristic numerical method, in which the positions of each joint is iteratively adjusted in a backward (from EE to base) and then forward (base to EE) manner. This is computationally cheaper and easier to implement than gradient methods [17], but under very high precision requirements FABRIK struggles to converge.
FABRIK is also very versatile, with a lot of research done into variations for solving different problems. It can be used to solve for multiple EEs. It has also been extended to solve in scenarios where the kinematics are not like a tree, but instead have loops.
Sequential Quadratic Programming is a Lagrange-based iterative method [19] that naturally incorporates secondary optimisations. It optimises a non-linear scalar function, given a vector constraint. SQP converges faster than heuristic methods, but is sensitive to the initial guess. Xu *et al.* [16] combine FABRIK with SQP by having FABRIK generate the initial guess, and using SQP to converge to a solution.

Cyclic Coordinate Descent (CCD) is another heuristic numerical method, but is designed for serial kinematic chains and thus is difficult to effectively implemented for more generalised robotic systems. It also returns solutions that aren't typically considered natural [17].

Many other numerical methods exists, generally treating IK as a non-linear optimisation problem, and using algorithms such as Riemannian optimisation or the derivate-free optimisation [16]. These are not discussed because of their lack in popularity. Some research has also been done in using neural networks to perform parts of computation, thereby reducing reliance on knowledge of the robot's kinematic structure [16][20]. With the aim of extending Jacobian-based methods to soft robots, Fang *et al.* [21] implemented a neural network to learn the Jacobian computation, and translate the abstractions from simulation into actuation.

FABRIK is the preferred IK implementation, because it offers a very versatile foundation with fast convergence times. For the level of precision that the Braccio offers, FABRIK's difficulty with high precision is likely not a major concern. The particular implementation of FABRIK needed would have joint limit constraints and possibly secondary objectives.

2.5 Computer Vision

Considering that the desired kinematic mapping method is end-effector mapping, a suitable computer vision pipeline would return the cartesian coordinates of an operator's fingertips and the global coordinate of their hand. This rules out computer vision pipelines that perform gesture recognition, such as in the works of Reyes *et al.* [2] and Simão *et al.* [13]. This also narrows the available computer vision pipelines to those that use joints as the abstraction level, as opposed to other abstractions such as silhouettes, edges, blobs, et cetera [22].

Moeslund and Granum [22] make note of the common assumptions made in computer vision. Assumptions they list which are acceptable for this project are: *(i) The subject (human arm) remains inside the workspace. (ii) There is no camera motion. (iii) Only one subject (human arm) is in the workspace at the time. (iv) There is no occlusion of the subject. (v) The motion pattern (kinematics) of the subject is known. (vi) There are minimal changes to lighting. (vii) The background is static/uniform. (viii) The camera parameters are known.*
With these assumptions in mind, the robustness of the computer vision pipeline is not as significant of a priority as accuracy and speed.

Monocular depth ambiguity is a common challenge in computer vision, especially when setups with depth cameras and/or multiple cameras are expensive or difficult to configure. Drăgoi *et al.* [3] faced monocular depth ambiguity as one of the primary contributions to error in their teleoperation system. They used the apparent hand size (calculated via distance between landmarks) as a proxy for depth. Another common method for depth estimation is to use deep machine learning directly [23], such as with denoising diffusion models [24].

2.6 Network

Mahmoud *et al.* [25] discuss various cyber threats to teleoperation systems. Unencrypted networks are vulnerable to *eavesdropping*, *message modification*, and *false data injection*. Networks without authentication are vulnerable to man-in-the-middle attacks, such as *replay*, *delay*, message dropping, and *spoofing*.

The network security, performance and reliability (referred to as the "trilemma" by Gounine *et al.* [26]) are a significant concern in teleoperation systems, especially where teleoperation is implemented in medical applications such as telesurgery (teleoperation for surgery) [27]. Due to the very strict regulations surrounding it, telesurgery has its own dedicated software security frameworks, such as SecureSurgiNET [28]. SecureSurgiNET uses a central telesurgical authority (TSA) to manage and verify certificates in their specialised public key infrastructure (PKI), similar to the X.509 standard.

Boston Dynamic's Spot often serves as a cybersecurity benchmark, having very mature security which surpasses that of almost all commercial quadruped robots [29]. Spot's security implements encrypted communications (TLS 1.2+), authentication (X.509 and ES256), firmware integrity, and per-device unique encryption keys. A different popular approach to secure teleoperation is **Secure and Statistically Reliable UDP** (SSR-UDP), which is an adaptive information coding method that provides encrypted traffic, authentication, and best-effort retransmission [25].

Takanashi *et al.* [30] successfully develop an encrypted four-channel bilateral control scheme. They implemented **homomorphic encryption**, which allowed them to do some computation while the data remained encrypted. This improves security against eavesdropping at the recipient-side. This is promising for more complex control schemes, but has little benefits for the unilateral control scheme of this project.

Virtual Private Networks (VPNs) are a popular approach to general network security and privacy. Among VPN protocols, such as OpenVPN, Tailscale and ZeroTier, **WireGuard** has the best performance [31]. This advantage also extends to IoT devices [32]. On an ESP32, the resource consumption of WireGuard is within operational limits, but WireGuard's encryption algorithms introduce $\approx$ 600ms of one-way latency [33]. This makes it suitable for logging and telemetry contexts, but less so for realtime control.

Overall, leaning towards an approach similar to Spot and SSR-UDP best aligns with goals and constraints of the project.
Both approaches implement retransmitting failed data packets, either through a TCP-based protocol or best-effort statistical reliability. This feature can be dropped for this project, given the nature of the teleoperation; by the time a packet has failed and needs to retransmitted, its information about the operator's arm is outdated. This lends itself to a UDP-based protocol, where authentication and encryption remain of importance.

Chapter 3

Computer Vision

3.1 Tools

The computer vision pipeline of choice was inspired by Drăgoi *et al.* [3]. OpenCV (5.0.0.93) was used to extract the live video feed of the laptop's camera, which was then fed into Google's MediaPipe (1.0.1) Hand Landmark Detection (later referred to as just "MediaPipe" in this paper). MediaPipe Hand Landmark Detection provides monocular hand tracking and pose estimation, and is the successor to MediaPipe Hands, as MediaPipe Hands is now deprecated.

Other computer vision pipelines that were considered include Ultralytics' YOLO and MMPose. The **YOLO models** are extremely customisable and easily trainable, making them excellent for niche computer vision tasks such as forensics [34], and overall hand detection, given a good data set [35]. There are no pre-trained models for hand pose estimation, and the time constraints of this project did not allow for training a custom model. **MMPose** has pre-trained models for hand pose estimation, but isn't as accurate as MediaPipe for this specific task [36].

The subsystem was implemented using Python 3.12 on a laptop whose specifications are listed in Table 3.1. MediaPipe was configured to only track one hand, while all other settings were default where applicable. NumPy (2.5.3) was used for the more computationally heavy linear algebra.

Table 3.1: Laptop Specifications

Specification	**Name/Value**
CPU	Intel Core i7-8565U, 8 threads @ 1.80GHz
Memory	16GiB DDR4
Operating System	EndeavourOS Linux (Kernel 7.2.6-arch2-1, 64-bit)
Camera	1280x720p @ 30fps, 78.6° FOV

3.2 Calibration

Before determining the end-effector target from the pose estimation, there's a calibration phase. The operator places their hand at four coordinates of their workspace, which they would want to roughly map to Braccio's workspace, 300mm along each axis. In order, the calibration coordinates are

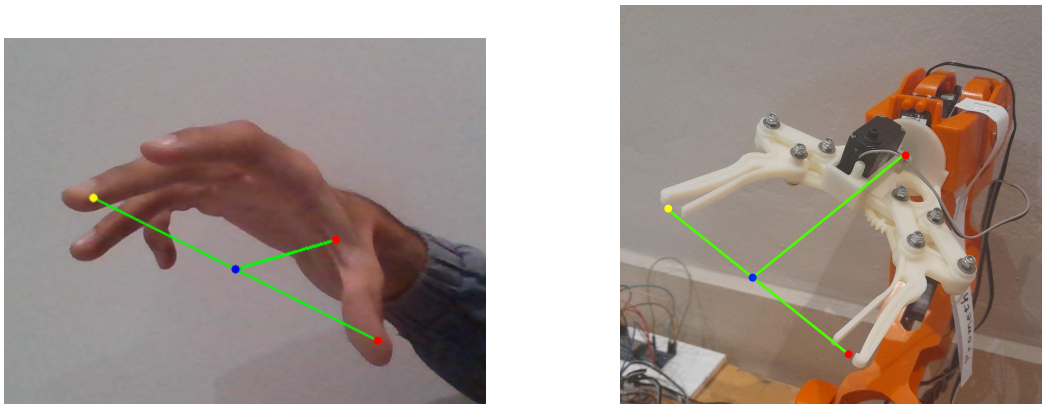
1. (0, 0, 0) - directly in front of the operator, corresponding to the base of the Braccio
2. (300mm, 0, 0) - to the right of the workspace
3. (0, 300mm, 0) - at the far end of the workspace, forward
4. (0, 0, 300mm) - at the top of the workspace

Using MediaPipe's interpretation of these coordinates, an offset and 3×3 scaling matrix is determined, which maps the workspace coordinates to MediaPipe's interpretation. The negative of the offset, and the pseudo-inverse of this matrix are then used to map future MediaPipe interpretations into workspace coordinates, for any hand pose inside the camera's view.

An attempt was made to incorporate the hand's apparent size (measured as the distance from the wrist to the base of the index finger) as a proxy for depth. The reciprocal of its size is roughly proportional to the distance away from the camera. This approximate distance, scaled by 1000 to have more weighting, was added to MediaPipe's interpretation of depth, to give a combined X-coordinate. This combined X-coordinate was used as the input for the calibration, as well as the hand pose estimation post-calibration. Even when using size as a proxy, the transformed depth estimation oscillated with 50 to 100mm of error, and was very sensitive to the orientation of the hand, as this would affect the perceived distance from the wrist to the base of the index finger. This was deemed too unreliable, so depth was excluded from the final implementation, forcing the Braccio to only move within the YZ-plane.

3.3 Target Generation

A "target" describes the desired position, direction, rotation and openness of the end effector. This information is extracted from the hand's wrist, thumb and middle-finger. The midpoint between the middle-finger and thumb is used as the target position. The unit vector from the wrist to the midpoint describes the direction. The unit vector from the midpoint to the middle-finger describes the rotation. Lastly, the distance between the thumb and middle-finger describes the openness. This interpretation of the hand pose can be seen in Figure 3.1, and was chosen because it was believed to be the most intuitive.



(a) Photo of human hand

(b) Photo of Braccio claw

Figure 3.1: Illustration of the target mapping from human hand to Braccio claw

The target is directly calculated from the workspace coordinates generated by transforming the MediaPipe coordinates. The X component of the position and direction are forcefully set to zero, to limit the target to the YZ-plane (forward and upward). The position is clamped to have Y and Z coordinates ≥ 0, to ensure that the target is always in front of the Braccio and also not misinterpreted as under the ground. If the position falls within 100mm of the origin, it is then projected outward onto a sphere with a radius of 100mm, ensuring the target isn't inside the Braccio. If the position was at the origin before being projected, it is instead defaulted to (0, 100mm, 0).
Unexpectedly, MediaPipe always interprets an upward facing palm (whenever the thumb is higher than the middle-finger) as having the thumb further from the camera, compared to the middle-finger. This is inaccurate in this context, because an operator showing an upward facing palm usually just intends for a clockwise rotation (facing the direction that the claw faces). To resolve this, the X component of the direction is negated if it's ever negative, forcing only positive X.
The direction and rotation are then normalised, before being handed over to the inverse kinematics solver along with position.

Even with the calibration described above, the range of the operator's openness varied significantly, sometimes ranging from 10mm to 50mm, and other times ranging from 20mm to 300mm, for different sessions. Because of this, the target's openness is calculated using an adaptive range conversion. An initial range of 30 to 40 is set, and whenever the openness exceeds the maximum or falls short of the minimum by 20%, the range is adjusted to fit the new value. This allows more consistent interpretation of the operator's hand's openness.
The valid actuation values for the Braccio's claw ranges from 10 (completely open) to 73 (completely closed). This is used to calculate the actuation that will best correspond to the operator's openness, using the range conversion in Equation 3.1.

$$\text{braccio openness} = 73 + (\text{hand openness} - \text{min}) \times \frac{10 - 73}{\text{max} - \text{min}} \tag{3.1}$$

3.4 Testing

This subsystem is benchmarked on the time taken to fetch a video frame, landmark the frame, and generate a target using the landmarks. This is done using the `monotonic()` function provided by the `time` module. Since MediaPipe processes frames asynchronously with the main thread, the timestamps provided to the MediaPipe thread are used as the initial time.

Data was generated during a teleoperation task, where a foam cube was picked up, rotated and dropped. A video of this task can be found on GitHub.

Chapter 4

Inverse Kinematics

4.1 Requirements

The inverse kinematics solver must be able to find a solution where one exists, or find a reasonable solution where no true solution exists. No true solution exists when the target is unreachable due to the kinematics of the serial manipulator or due to limited range of motion. The acceptance test procedures that test for these requirements are described in subsection 4.4.1.

Table 4.1: Requirements

ID	Description
RQ1	A true solution must be found where one exists.
RQ2	A reasonable solution must be found, where the kinematics prevent a true solution.
RQ3	A reasonable solution must be found, where a limited range of motion prevents a true solution.

4.2 Algorithm

The inverse kinematics subsystem uses the FABRIK algorithm. In particular, it implements an adaptation of FABRIK described by Moreno *et al.* [37], which incorporates movement for more types of joints (not just ball joints, like in the original FABRIK), as well as constraints on range of motion. An illustration of each joint's behaviour can be seen in Figure 4.1. The pseudocode for the overarching algorithm can be seen in Algorithm 1, while the pseudocode for the forward phase of each type of joint can be seen in Appendix C.

This implementation uses an array of Joint structs to describe the kinematics and positions/orientations of a serial manipulator, such as the Braccio. The members of a Joint struct are described in Table 4.2. Like with normal FABRIK [38], the algorithm iterates through the array, adjusting positions of joints under the constraints of each one's prior joint. This process is repeated for both a forward-reaching and backward-reaching stage. The adaptation of FABRIK restricts where successive joints can be positioned based on the type of joint that came before, and then corrects the position based on that prior joint's range of motion, such as [-90°, 90°].

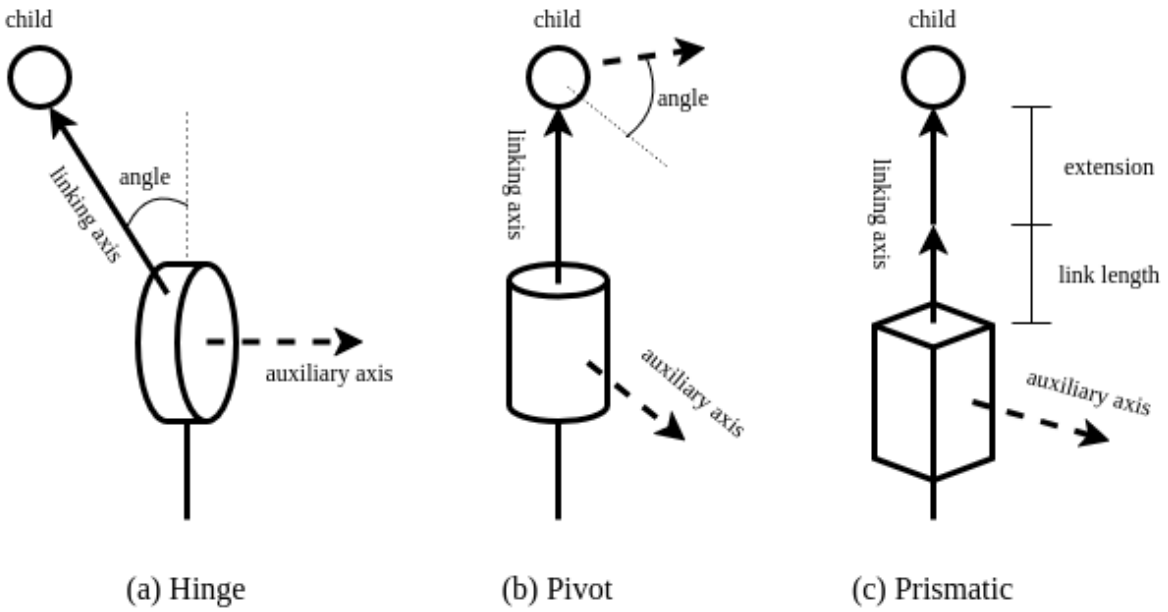


Figure 4.1: Illustration of different types of joints

Table 4.2: Joint members

Member Name	**Type**	**Description**
`joint_type`	Enum	The type of motion this joint is limited to, such as ”Hinge”
`len_link`	Float	Fixed distance from this joint to its child[1]
`pos`	Vector3	Absolute position in space
`axis_link`	Vector3	Unit vector pointing to its child[1], referred to as the Z-axis by Moreno *et al.*
`axis_aux`	Vector3	Perpendicular vector about which a hinge rotates, referred to as the X-axis by Moreno *et al.*
`value`	Float	The angle of rotation of a hinge or pivot, or the extension of a prismatic joint
`limited`	Boolean	Whether or not the joint’s range of motion is limited, hence limiting `value`
`min`	Float	The lower bound of the range of motion
`max`	Float	The upper bound of the range of motion

In the adapted FABRIK algorithm, joints always need another joint before and after them, in order to be adjusted during both the forward and backward-reaching phases. This necessitates having dummy joints at the start and end of the array, which behave as the mounting base and target respectively.
In this subsystem’s implementation, a new ”reference” joint type is introduced to resolve this issue. This joint type does not inherently displace subsequent joints. Its behaviour is equivalent to a prismatic joint whose linking-length and extension are fixed at zero.
A third instance of this joint type is necessary to model the end-effector. In the case of a serial manipulator with only one functional joint, that joint would simply be thrown back and forth between the base and target, never having its `value` be adjusted towards a solution. This is because a joint’s `value` is only ever adjusted when a neighbouring joint is moved, and that movement is dependent on it. This is resolved by adding another reference joint before the target joint, which will undergo the FABRIK process.

[1] In this paper, the ”child” of a joint is another joint which is directly attached to and manipulated by it.

Algorithm 1 FABRIK Algorithm

Require: An array J of joints where $J[0]$ is the base, $J[n-2]$ is the end-effector, and $J[n-1]$ is the target, an acceptable squared error e, and a maximum number of forward-backward iterations m

Ensure: The end-effector is within e of the target, or iterations reach maximum

function FABRIK(J, e, m)
  // Loop until error reduced or iterations maxed
  $error \leftarrow |J[n-1].\vec{pos} - J[n-2].\vec{pos}|^2$
  $error \leftarrow error + |J[n-1].\vec{axis}_{link} - J[n-2].\vec{axis}_{link}|^2$
  $error \leftarrow error + |J[n-1].\vec{axis}_{aux} - J[n-2].\vec{axis}_{aux}|^2$
  $i \leftarrow 0$
  while $i < m$ **and** $error > e$ **do**
    // Forward reaching
    for $j \leftarrow (n-2)$ to 1 **do**
      if $J[j+1]$ is a reference **then**
        ForwardReference(J, j)
      else if $J[j+1]$ is a hinge **then**
        ForwardHinge(J, j)
      else if $J[j+1]$ is a pivot **then**
        ForwardPivot(J, j)
      else if $J[j+1]$ is prismatic **then**
        ForwardPrismatic(J, j)
      end if
    end for

    // Backward reaching
    for $j \leftarrow 1$ to $(n-2)$ **do**
      if $J[j-1]$ is a reference **then**
        BackwardReference(J, j)
      else if $J[j-1]$ is a hinge **then**
        BackwardHinge(J, j)
      else if $J[j-1]$ is a pivot **then**
        BackwardPivot(J, j)
      else if $J[j-1]$ is prismatic **then**
        BackwardPrismatic(J, j)
      end if
    end for

    // End iteration
    $error \leftarrow |J[n-1].\vec{pos} - J[n-2].\vec{pos}|^2$
    $error \leftarrow error + |J[n-1].\vec{axis}_{link} - J[n-2].\vec{axis}_{link}|^2$
    $error \leftarrow error + |J[n-1].\vec{axis}_{aux} - J[n-2].\vec{axis}_{aux}|^2$
    $i \leftarrow i + 1$
  end while
end function

Moreno *et al.* don't consider an edge-case in their adapted FABRIK algorithm, in which the last functional joint is a pivot. This case is relevant to the Braccio, as its wrist joint is modelled by a pivot. During the backwards-reaching phase of such a scenario, the auxiliary axis of the end-effector would be chosen such that, if it were a hinge, it would more easily accommodate for its child. This assumption doesn't hold for such a case and leads to less desirable results. This subsystem instead handles this case uniquely, and chooses an auxiliary axis that best aligns with the target's auxiliary axis. It does so by projecting the target's auxiliary axis onto the plane defined by the pivot's linking axis, and all subsequent processing is identical to the original algorithm; check for zero magnitude, normalise, and apply bounds on range of motion.

4.3 Implementation for the Braccio

The Braccio was modelled as having five functional joints, and hence an array of eight Joint structs (to include the base, end-effector, and target). The configuration for each joint is shown in Table 4.3. The shoulder joint's range of motion was confined to 0 degrees, aligning with the 2D dimensional restriction discussed in section 3.3. This makes the joint redundant, but it was kept for the sake of being more explicit, and deemed to have negligible effect on the overall performance of the system.

Each time an inverse kinematics solution needs to be found, the target position, direction, and rotation from the computer vision subsystem is parsed as the target position, linking axis, and auxiliary axis, respectively. The auxiliary axis is then projected onto the plane defined by the linking axis and normalised again, to ensure that it's perpendicular to the linking axis. In edge cases with zero magnitude, the previous inverse kinematics solution is returned. After setting the target, the adapted FABRIK algorithm is run, starting at the previous solution, for up to 10,000 iterations while aiming for a squared error under 1mm^2. The 10,000 iteration limit was experimentally selected by testing the computation time for different limits in scenarios that struggled to converge. The solver returns the `value`s of the functional joints, which then all undergo a mapping to the Braccio's native angular measurements, giving five setpoints.

Considering the computationally heavy nature of a numerical process, the algorithm was implemented in Rust (1.99.0) as opposed to Python. The Maturin (1.15.0) package was used to compile the Rust code into a Python module, giving the Python main loop access to inverse kinematics function calls and return values. This subsystem was run on the same laptop described in Table 3.1.

Table 4.3: Braccio joint configurations

Member	**Base**	**Shoulder**	**Arm 1**	**Arm 2**	**Arm 3**	**Wrist**	**EE**	**Target**
`joint_type`	Reference	Pivot	Hinge	Hinge	Hinge	Pivot	Reference	Reference
`len_link`	N/A	68mm	123mm	123mm	62mm	125mm	N/A	N/A
`pos`	0, 0, 3mm	-	-	-	-	-	-	position
`axis_link`	0, 0, 1mm	-	-	-	-	-	-	direction
`axis_aux`	1mm, 0, 0	-	-	-	-	-	-	rotation
`value`	N/A	-	-	-	-	-	N/A	N/A
`limited`	N/A	true	true	true	true	true	N/A	N/A
`min`	N/A	0°	-75°	-90°	-90°	-90°	N/A	N/A
`max`	N/A	0°	75°	90°	90°	90°	N/A	N/A

4.4 Testing

4.4.1 Acceptance Test Procedures

The inverse kinematics solver is acceptance-tested in isolation from the Braccio-specific implementation of the subsystem. Each of the main three joint types undergo three scenarios in which they are the only joint of a serial manipulator (1 degree of freedom):

1. The joint's range of motion is not constrained, and the target is perfectly reachable.
2. The joint's range of motion is not constrained, but the target is unreachable regardless, due to the kinematics of the joint.
3. The joint's range of motion is constrained, and the target would be reachable if not for the constrain.

This creates a total of nine ATPs. Each ATP is given up to 100 iterations, aiming for less than 1mm^2 of error. An ATP is passed if the returned solution (`value` associated with the Joint) is within 0.01 degrees or millimetres of the expected solution. The exact set of scenarios used to test the inverse kinematics solver can be found in Appendix D.

4.4.2 Benchmarking

This subsystem is benchmarked on the time taken to attempt to find an inverse kinematics solution, the number of iterations in attempts, and the remaining error after attempts. Time is measured using the `monotonic()` function from the `time` module in Python. The total error encompasses the error in the position, direction and rotation, in a sum-of-squares relationship.

Data was generated during the same teleoperation task used to generate data for the computer vision subsystem. A video of this task can be found on GitHub.

Chapter 5

Network

5.1 Internet Protocol

5.1.1 Protocol

One of two schemes can be adopted; either the operator-side sends packets only when a new target is determined, or the operator-side sends packets continuously, regardless of if there's a change. The former reduces network traffic, while the latter implicitly gives the slave-side information about the network's reliability. In terms of the former, the reduction in network traffic isn't significant, if we assume the operator is in motion for a significant portion of the total teleoperation session. The latter allows the Braccio to react to network issues, that may have been caused by jamming or DOS attacks, making this scheme preferred.

Instead of building up a protocol that handles authentication and encryption from scratch, DTLS was implemented. The ESP32 was configured as a DTLS server, exposing port 42069 to receive the actuation values.

For authentication, the choice was between X.509 certificates or preshared keys (PSK). X.509 certificates are generally more secure, and allow the permissions/authentication of a participant to be revoked (although managing multiple operators' permissions is outside of the scope for this project). Processing X.509 certificates is resource intensive, which makes the ESP32 more vulnerable to DoS attacks while it's waiting for a valid handshake. PSKs have much less overhead during the initial handshake, making it the implementation of choice.

DTLS was configured to use a 128-bit PSK, and enable Hello Verify cookie exchange to further reduce the risk of DoS attacks.
It was also configured to end the current session if no valid packets are received within 1 second of the previous valid packet. This allows DTLS to be configured to reject any new sessions, while a session is already active, without the risk of the operator-side failing without a sending a notification to close the session.
The chosen symmetric encryption algorithm was AES-128, with GCM to check for tampering, because AES is the encryption algorithm the ESP32 has hardware accelerators for [39].

DTLS accepts a sliding window of past packets whose sequence numbers were not already received, thereby rejecting packets that were already received or are older than the sliding window [40]. The ESP32 shouldn't accept any packets older than the latest, so a manual implementation of unsigned 16-bit sequence numbers was implemented in addition to DTLS's. This manual implementation enforces that only later sequence numbers are accepted by checking that the difference between the received and previous sequence number is positive. This method eventually wraps around, and is vulnerable for some sequence numbers. It uses a 16-bit unsigned integer for the sequence numbers, which means the wraparound happens every $2^{16} = 65,536$ packets, and packets as recent as $2^{15} - 1 = 32,767$ instances ago will pass this check. Packets as old as that are rejected by DTLS's sliding window, thereby covering this vulnerability.

5.1.2 Payload Format

The payload contains the 16-bit unsigned sequence number, as well as six 32-bit floating point numbers that're the setpoints for the Braccio's degrees of freedom. These setpoints consist of the five setpoints from the inverse kinematics subsystem, and the openness of the claw, calculated in the computer vision subsystem. Bytes are ordered in little-endianness, because this is the native format of the laptop and Arduino.

Table 5.1: Internet Payload Format

Address	Content
0x00 - 0x01	Unsigned sequence number
0x02 - 0x05	Shoulder joint setpoint
0x06 - 0x09	Arm 1 joint setpoint
...	...
0x12 - 0x15	Wrist joint setpoint
0x16 - 0x19	Claw openness setpoint

5.1.3 Tools

DTLS on the ESP32 was implemented using MbedTLS. This limited the protocol to its 1.2 version, as MbedTLS doesn't currently support TLS/DTLS1.3.

The PSK was generated using OpenSSL's (3.6.4) `rand` command.

The operator-side of the network runs on the same laptop as the computer vision and inverse kinematics subsystems. This was implemented using bindings to C types and functions via the `ctypes` module, and interfacing with OpenSSL.

5.2 Communication with Arduino Uno

5.2.1 Protocol

The ESP32 and Arduino are serially connected through a TXS0108E logic level shifter, to handle the 5V/3.3V conversion. The communication protocol between the ESP32 and Arduino is UART, because the ATmega328P has hardware for UART that allows it to receive bytes asynchronously [41], unlike with SPI and I2C, which would occupy the Arduino's main thread during reception.

The highest baud rate that successfully transmits data, while also being a divisor of the Arduino's 16MHz clock, was experimentally determined to be 640kBd. Since the ESP32, Arduino, and logic level shifter are all rated above this baud rate, a possible cause of this limit is the breadboard's parasitic capacitance. The UART configuration sends 8 data bits per frame, with no parity bit, and 1 stop bit, which are all defaults on the Arduino.

5.2.2 Payload Format

Since UART is just a byte stream, the starting point of a payload needs to be indicated/marked. This is done with a 2-byte header. The first two bytes of the payload are always 0xC0 and 0xDE, and the Arduino waits for these two bytes before parsing the rest of the payload.
The data of the payload is the 24 bytes of setpoints (little-endian), followed by CRC16 bytes to check the payload integrity. The CRC is generated by the ESP32 using the GENIBUS configuration.

Table 5.2: UART Payload Format

Address	**Content**
0x00 - 0x01	Header: 0xC0, 0xDE
0x02 - 0x05	Shoulder joint setpoint
...	...
0x16 - 0x19	Claw openness setpoint
0x1A - 0x1B	CRC16

5.3 Testing

5.3.1 Penetration Testing

The following tests were performed to verify the implemented security:

- Sending an HTTP request, to check that the Braccio doesn't respond to random protocols
- Sending a valid DTLS payload, but with the incorrect key
- Disconnecting for over 1 second, and trying to send packets again
- Confirming that packets were encrypted, using Wireshark

5.3.2 Benchmarking

This subsystem is benchmarked on the time taken:

- to transmit a packet on the operator-side, measured with `monotonic()`
- for a packet to reach the ESP32, measured by pinging the ESP32 and assuming half of the round trip time
- to receive and validate a packet on the ESP32-side, measured with `esp_timer_get_time()` from the `esp_timer.h` library
- to transmit the UART payload, determined by calculation

Data for the operator-side's transmission latency was generated during the same teleoperation task used to generate data for the inverse kinematics and computer vision subsystems. A video of this task can be found on GitHub.
Data for the ESP32-side's reception and validation latency was generated by sending a constant stream of valid packets to the ESP32, while the ESP32 logged the latency to PlatformIO's serial monitor.

Chapter 6

Actuation

6.1 Braccio Control Library

Arduino's default library for controlling the Braccio [42] was deemed unfit for this project. The API function to set the Braccio's position increments/decrements the angle of each motor by 1 degree repeatedly, until all motors are at their setpoints. Between each iteration is a delay that is chosen when the function is called.
This implementation introduces several problems:

- The function is blocking, and thus can't change trajectory if a new setpoint is received mid-motion.
- The delay between each iteration is fixed. At moderate speeds, there's a large acceleration at the start and end of the motion, causing the Braccio to jerk.
- Some motors will reach their setpoint before others, making the end-effector's motion not smooth.
- The most precision that can be achieved is 1 degree, not making use of the servo motors full precision. This also makes movement look jittery (quantised) when moving slowly.

A new library was implemented that uses a cooperative multitasking paradigm, where the motors approach their setpoints gradually over multiple function calls, allowing other processes to occur in between. The setpoint is updated with a function call, and the motors are stepped with a different function call.

The function to step the motors determines the new position of the motors using 2nd-order exponential smoothing, shown in Equation 6.1. $\vec{r}$ is the setpoint, $\vec{p}$ is an intermediate variable, $\vec{q}$ is the angle of the motors, and α is the smoothing factor. This way of calculating the next motor positions was chosen because it mimicked a physical system with a mass and damping, reducing jerking at both the start and end of the motion.

$$\begin{aligned} \vec{p}_{i+1} &= (1-\alpha)\vec{p}_i + \alpha\vec{r} \\ \vec{q}_{i+1} &= (1-\alpha)\vec{q}_i + \alpha\vec{p} \end{aligned} \tag{6.1}$$

Choosing a low enough value for α helps with both reducing jerk, and filtering out noise in the setpoints. The desired α can be calculated using a desired time-constant τ, with an RC low-pass filter approximation [43], shown in Equation 6.2. The period T at which the motors are stepped is not necessarily constant, so the `millis()` function is used to calculate the time since the last step. A suitable value for τ that filtered out enough noise was experimentally determined to be 100ms.

$$\alpha = \frac{1}{1 + \frac{\tau}{T}} \tag{6.2}$$

To command the servo motors into a particular angle, the `writeMicroseconds()` function from the `Servo` library is used to directly control the PWM pulse width, thereby giving more precision. The Braccio uses two types of servo motors, both of which accept a pulse width from 500 to 2500 microseconds and have a range of motion of 180 degrees [44][45]. Desired angles $\vec{q}$ from the exponential smoothing are converted to pulse widths $\vec{w}$ using a range mapping, shown in Equation 6.3.

$$\vec{w} = 500 + (\vec{q} - 0) \times \frac{2500 - 500}{180 - 0} \tag{6.3}$$

The new library also implements soft-start; a feature of the old library that gradually increases power to the servo motors when the Arduino is turned on, as opposed to letting the servo motors jolt into an initial position.

6.2 Main Loop

The Arduino's main loop continuously gathers bytes from the UART buffer, and searches for a complete payload. If a complete payload is found and verified with CRC16, then the setpoints for the motors are updated. If only a partial payload is found, that partial portion is kept for the next iteration of the main loop.
The Arduino also keeps track of when the latest setpoint update was. If the setpoints have not been updated for over 1 second, then whatever teleoperation task was in progress is likely compromised and so the Arduino resumes a "safe" protocol; it reduces the speed of the Braccio by setting τ = 300ms, and sets the setpoint to a retracted position with the claw open. Once a new setpoint is received, the time constant is set back to τ = 100ms and operation resumes as normal.
At the end of every iteration of the main loop, the motors are stepped, thereby approaching the setpoint.

6.3 Testing

This subsystem is benchmarked on the time taken to settle at the setpoint. This is measured by video capturing the Braccio during a 90°step response, using a 30fps camera. This video can be found on GitHub.

Chapter 7

Results

All raw data can be found on GitHub.

7.1 Latency

The latency of the system due to each stage of the pipeline is shown in Figure 7.1 as a box and whisker plot. The mean and standard deviation of these latencies, as well as the total are shown in Table 7.1. The step response of the Braccio, approximated to 10° steps is shown in Figure 7.2, along with a spline fit using $\lambda = 2$ to help visualise the step.

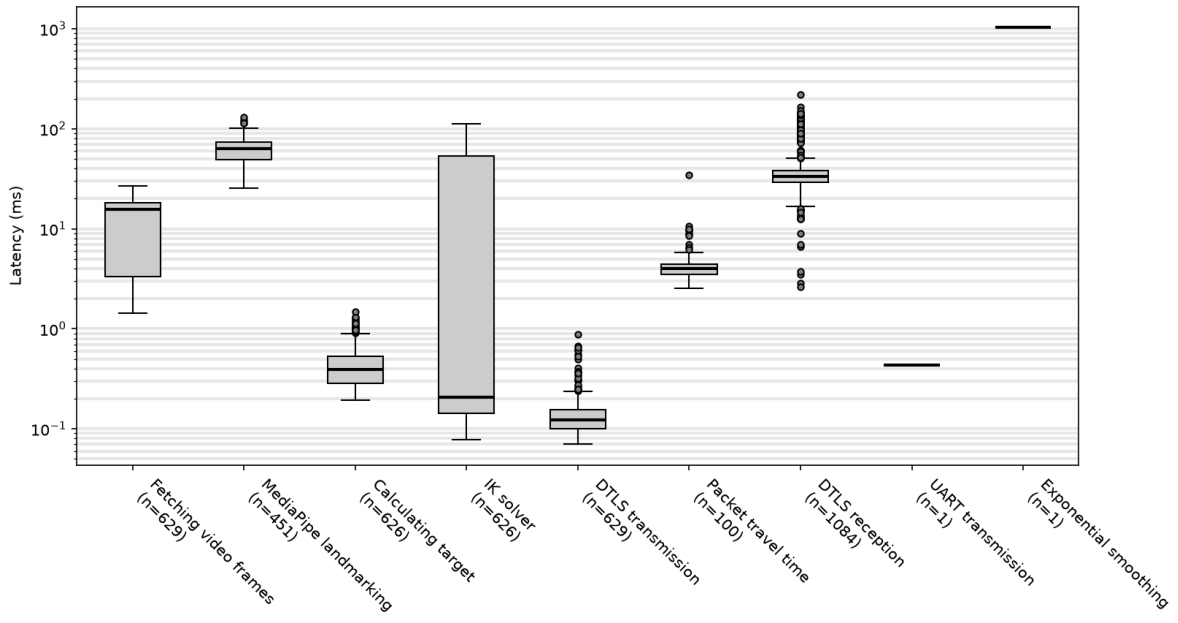


Figure 7.1: Box and Whisker plot of latency contributed by each stage

Table 7.1: Table of latency contributed by each stage

Stage	Mean (ms)	Standard Dev. (ms)	Number of Samples	Percentage of Total
Fetching video frames	12.3	7.47	629	1.05%
MediaPipe landmarking	61	18.7	451	5.21%
Calculating target	0.438	0.199	626	0.0375%
IK solver	16.9	28.2	626	1.45%
DTLS transmission	0.139	0.0724	629	0.0119%
Packet travel time	4.7	3.42	100	0.402%
DTLS reception	40.4	24.4	1084	3.45%
UART transmission	0.438	N/A	N/A	0.0374%
Exponential smoothing	1,030	N/A	N/A	88.3%
Total	1,170	42.5	N/A	100%

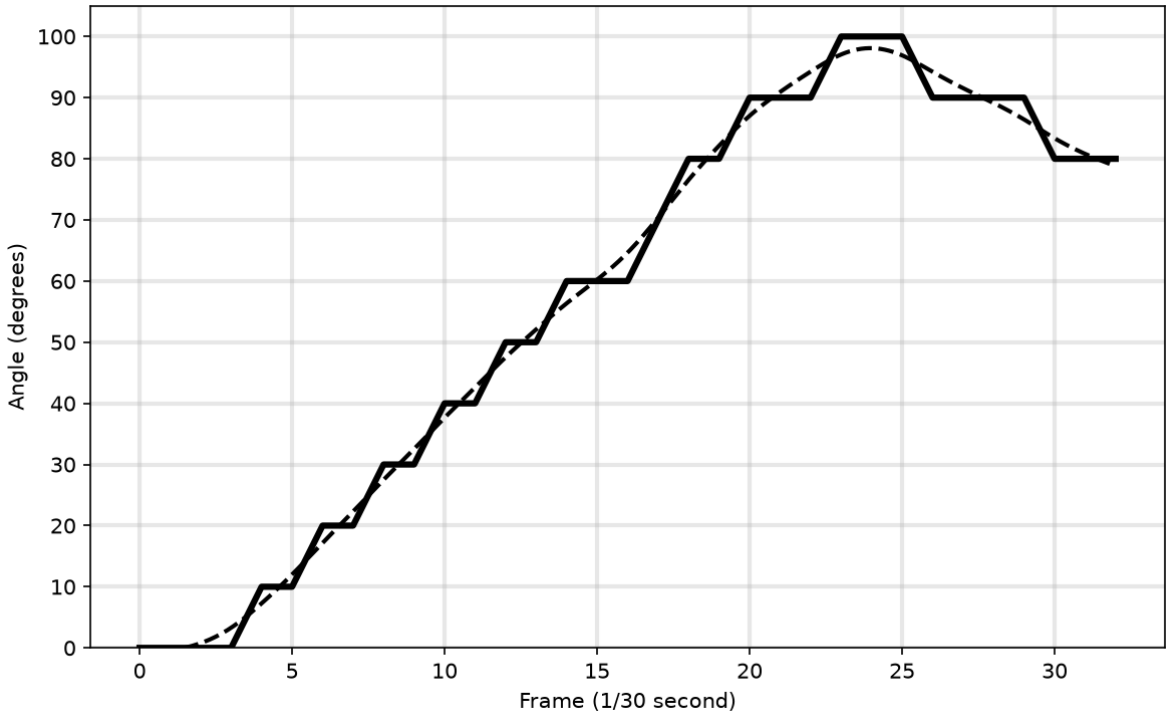


Figure 7.2: Braccio arm step response

7.2 Inverse Kinematics Solver

The remaining error over time, after running the inverse kinematics solver, is shown in Figure 7.3, while the latency and error per number of iterations is shown in Figure 7.4 and Figure 7.5 respectively.

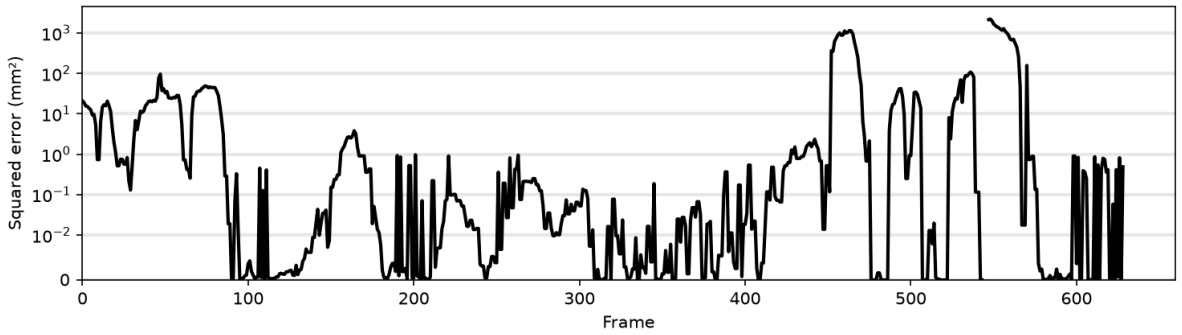


Figure 7.3: Squared error throughout the teleoperation task

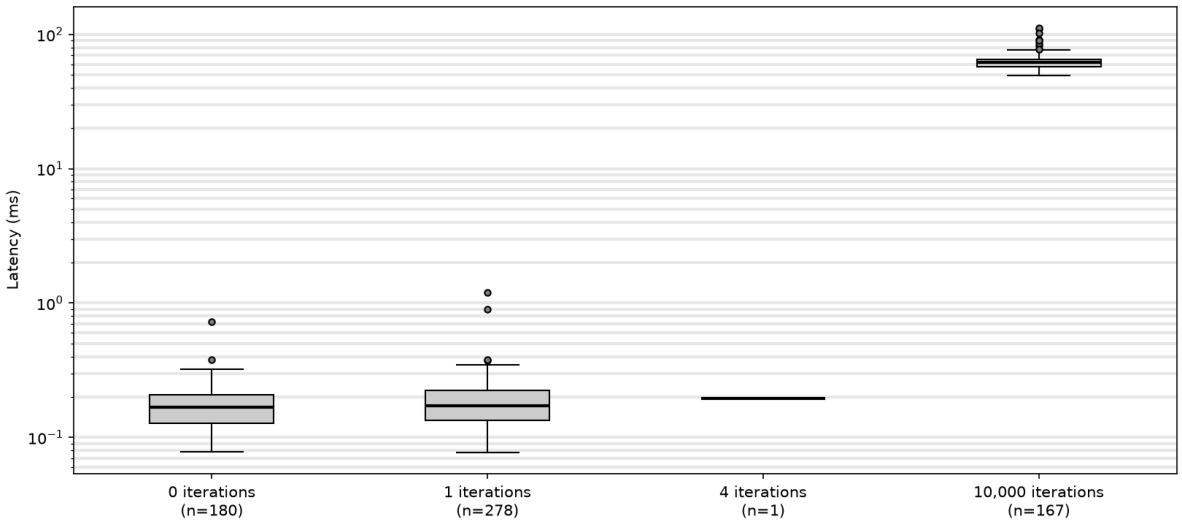


Figure 7.4: Latency per number of iterations

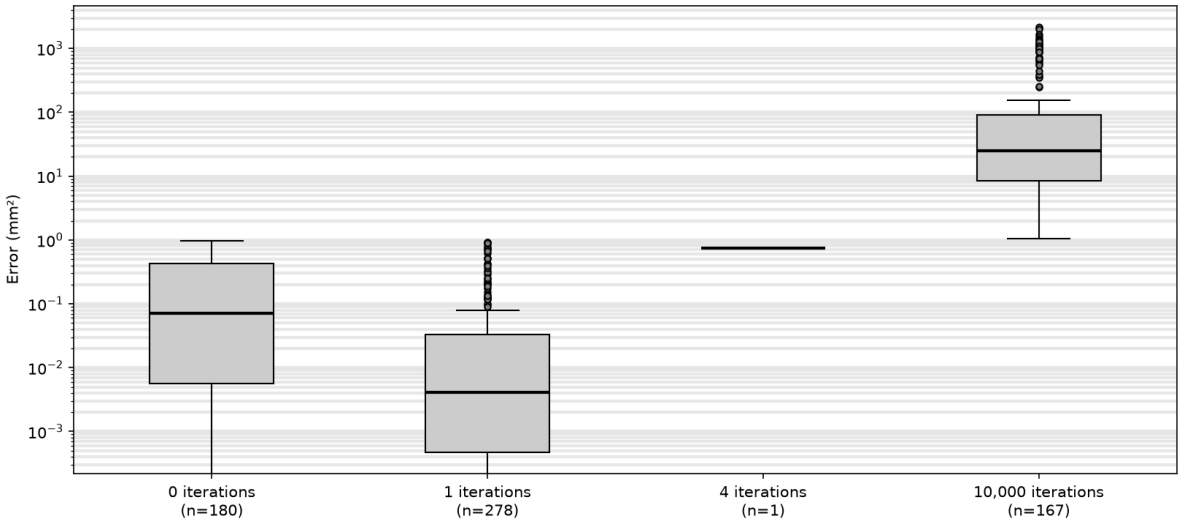


Figure 7.5: Squared error per number of iterations

Chapter 8

Discussion

8.1 Latency

As seen in Figure 7.1 and Table 7.1, the primary contributor of latency in the overall system is the exponential smoothing, contributing 88.3% of the total latency. This brings the total latency to 1.17 ± 0.0425 seconds.

This is significantly slower than other teleoperation systems. Mongardi *et al.* [1] achieve a total latency of 300ms, while Chen *et al.* [5] make their robotic manipulator have a motion-latency of only 10ms. However, being faster has a risk of more aggressive movements, which can be hazardous when dealing with industrial robotic manipulators, like in the case of TeleopLab [5]. The trade-off between speed and safety remains to be investigated.

The latency due to the networking subsystem (DTLS and UART communication) is 45.677ms, which is comparable to MediaPipe's landmarking latency of 61ms. Excluding the exponential smoothing, the networking subsystem contributes 33.51% to the remaining latency. This would be a significant portion. The Braccio was controlled over a relatively fast home network, making the packet travel-time smaller than what it would be in a practical application. The latency due to packets having to travel through the internet will vary significantly, depending on the distance between the operator and slave, reaching latencies upward of 150ms [46]. This is a factor that could not be expressed in the experiments of the project, and may dwarf the latency of the encryption and decryption (encompassed by the transmission and reception).

The step response of the Braccio has an overshoot of about 10°. Exponential smoothing can't inherently introduce overshoot, so this behaviour likely comes from the inertia of the Braccio's arm.
The settling point is also below the setpoint. Each linkage in the Braccio isn't held tightly in position and has a bit of wiggle room. This means the arm is able to "sag" due to the effects of gravity, which appears to happen in the video footage.

8.2 Inverse Kinematics Solver

The IK solver did not return a variety of iterations. Instead, the algorithm either:

- Used no iterations (occurred 180 times)
- Used 1 iteration (occurred 278 times)
- Used 4 iterations (an outlier, occurring once)
- Used the maximum 10,000 iterations (occurred 167 times)

From the performance of the overall system and the fact that the IK solver passes all ATPs, it's clear that the algorithm **does** return reasonable solutions. This implies that the algorithm rarely needs more than one iteration to find a solution, otherwise struggles to converge (either due to FABRIK's difficulty with precision or impossible target).

This is supported by the behaviour of the total error throughout the teleoperation task, seen in Figure 7.3. There is no clear convergence behaviour (such as a decaying exponential) that would suggest that the algorithm needs more than 10,000 iterations to converge. Instead, the error fluctuates, possibly due to the nature of the target being fed into the algorithm.
Because the target encompasses both position and direction, impossible targets are easy to find, hence yielding errors that can't be reduced. This can be seen when the operator places their hand near the edge of the Braccio's range of motion, and faces it downwards. For the Braccio to meet the positional requirement, it must extend its arm to the maximum length, but this causes the directional requirement to be failed.

For shorter amounts of iterations, the IK solver creates significantly less latency; around 0.2ms, which can be seen in Figure 7.4. This is on par with other IK solvers, where solution times ranges from 0.03ms to 3.45ms [47]. This comparison isn't completely fair, because other IK solvers typically aim for more precision (10^{-6}mm). After one iteration, the current implementation typically reduces the error below 0.1mm^2, as shown in Figure 7.5.
The current implementation would benefit from checking if convergence is failing and then deciding to terminate early.

8.3 Cybersecurity

The current implementation shows some successful defences against cyber attacks. In practice, this implementation is far from a secure system. Current vulnerabilities include, but are not limited to:

1. Side channel attacks
2. DoS attacks (untested)
3. Jamming
4. Physical attacks, such as glitch attacks
5. Reprogramming or reading the firmware
6. Any vulnerabilities on the operator-side device
7. Social engineering

Chapter 9

Conclusions

A full teleoperation pipeline with non-intrusive user readings and security conscious networking was successfully implemented, although with many limitations.

The final implementation failed to make use of the third dimension of space, thereby limiting the range of tasks the teleoperated slave can accomplish.

The total latency of the slave's response was slower than implementations in existing literature, primarily due to the low-pass filtering done to the Braccio's movements. The inverse kinematics solver has the potential to match the speed of other implementations, but suffers from impossible targets.

The cybersecurity conscious networking subsystem did not significantly impede on the overall system's performance, demonstrating a way to partially secure network traffic in resource constrained systems, such as an ESP32. Many other vulnerabilities still exist.

Chapter 10

Recommendations

Adding depth to the teleoperation is a natural next step, as only the computer vision subsystem would require significant modification. This can be done by using a depth camera, as opposed to the native laptop camera, or introducing a second camera at an orthogonal perspective. In the case of a depth camera, MediaPipe doesn't natively support depth data, so a separate algorithm/model would be required to create a depth map.

The current teleoperation system is still vulnerable to many types of cyber attacks. Implementing more cybersecurity measures would greatly improve the safety of the system. A way to improve the security would be to use the ESP32's flash encryption. The ESP32 and Arduino are also able to be bricked, such that their firmware can't be overwritten; a feature more for commercial release as opposed to research. Performing more thorough and comprehensive tests on the existing implementation, as well as any new features, would help improve confidence in how the attack surface is reduced.

This project only evaluates the kinematic mapping software with functional metrics, but does not provide direct insight into its intuitiveness. I would like to see the kinematic mapping software evaluated on intuitiveness, likely using the *completion rate* and *completion time* of teleoperation tasks, performed by novice operators. These metrics are used to evaluate intuitiveness in other teleoperation systems [3][9][10].

Bibliography

[1] A. Mongardi, F. Rossi, A. Prestia, D. Demarchi, and P. M. Ros, "Live demonstration: A wearable armband for real-time control of multi-dof robotic actuators," in *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2023, pp. 1–1. [Online]. Available: https://doi.org/10.1109/ISCAS46773.2023.10181982

[2] A. Reyes, S. Reinhardt, T. Wise, N. Rawashdeh, and S. Paheding, "Gesture controlled collaborative robot arm and lab kit," in *2022 CIEC*, 2024. [Online]. Available: https://peer.asee.org/45693.pdf

[3] M.-V. Drăgoi, A.-V. Frimu, A. Postelnicu, R.-A. Puiu, G. Petrea, and A. Hank, "Interactive teleoperation of an articulated robotic arm using vision-based human hand tracking," *Biomimetics*, vol. 11, no. 2, 2026. [Online]. Available: https://doi.org/10.3390/biomimetics11020151

[4] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, "Mediapipe hands: On-device real-time hand tracking," 2020. [Online]. Available: https://doi.org/10.48550/arXiv.2006.10214

[5] Z. Chen, Y. J. Yoon, R. Bautista-Montesano, Z. Zhao, A. Mandlekar, and J. Liu, "Teleoplab: Accessible and intuitive teleoperation of a robotic manipulator for remote labs," 2025. [Online]. Available: https://arxiv.org/abs/2509.05547

[6] S. Dass, W. Ai, Y. Jiang, S. Singh, J. Hu, R. Zhang, P. Stone, B. Abbatematteo, and R. Martín-Martín, "Telemoma: A modular and versatile teleoperation system for mobile manipulation," 2024. [Online]. Available: https://arxiv.org/abs/2403.07869

[7] MoveIt!, "Trac-ik kinematics solver." [Online]. Available: https://docs.ros.org/en/kinetic/api/moveit_tutorials/html/doc/trac_ik/trac_ik_tutorial.html

[8] Z. Wang, I. Reed, and A. M. Fey, "Toward intuitive teleoperation in surgery: Human-centric evaluation of teleoperation algorithms for robotic needle steering," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 5799–5806. [Online]. Available: https://doi.org/10.1109/ICRA.2018.8460729

[9] C. Meeker, T. Rasmussen, and M. Ciocarlie, "Intuitive hand teleoperation by novice operators using a continuous teleoperation subspace," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 5821–5827. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8460506

[10] C. Meeker, M. Haas-Heger, and M. Ciocarlie, "A continuous teleoperation subspace with empirical and algorithmic mapping algorithms for nonanthropomorphic hands," *IEEE Transactions on Automation Science and Engineering*, vol. 19, no. 1, pp. 373–386, 2022. [Online]. Available: https://doi.org/10.1109/TASE.2020.3035156

[11] R. Li, H. Wang, and Z. Liu, "Survey on mapping human hand motion to robotic hands for teleoperation," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 32, no. 5, pp. 2647–2665, 2022. [Online]. Available: https://doi.org/10.1109/TCSVT.2021.3057992

[12] Y. Su, X. Chen, T. Zhou, C. Pretty, and G. Chase, "Mixed reality-integrated 3d/2d vision mapping for intuitive teleoperation of mobile manipulator," *Robotics and Computer-Integrated Manufacturing*, vol. 77, p. 102332, 2022. [Online]. Available: https://doi.org/10.1016/j.rcim.2022.102332

[13] M. A. Simão, O. Gibaru, and P. Neto, "Online recognition of incomplete gesture data to interface collaborative robots," *IEEE Transactions on Industrial Electronics*, vol. 66, no. 12, pp. 9372–9382, 2019. [Online]. Available: https://doi.org/10.1109/TIE.2019.2891449

[14] S. Patong and N. Suresh, "Robotic motion planning using the jacobian matrix-based inverse kinematics algorithm," *Association Journal of Interdisciplinary Technics in Engineering Mechanics*, vol. 3, no. 3, pp. 14–21, 2025. [Online]. Available: https://ajitem.org/index.php/journal/article/view/EM33003

[15] S.-O. Park, M. C. Lee, and J. Kim, "Trajectory planning with collision avoidance for redundant robots using jacobian and artificial potential field-based real-time inverse kinematics," *International Journal of Control, Automation and Systems*, vol. 18, pp. 2095–2107, 2020. [Online]. Available: https://doi.org/10.1007/s12555-019-0076-7

[16] Z. Xu, Y. Li, X. Yang, Z. Zhao, J. Zhao, and H. Liu, "A combined inverse kinematics algorithm using fabrik with optimization," *Journal of Intelligent & Robotic Systems*, vol. 108, no. 62, 2023. [Online]. Available: https://doi.org/10.1007/s10846-023-01895-2

[17] A. Aristidou, Y. Chrysanthou, and J. Lasenby, "Extending fabrik with model constraints," *Computer Animation & Virtual Worlds*, vol. 27, pp. 35–57, 2015. [Online]. Available: https://doi.org/10.1002/cav.1630

[18] M. Kelemen, I. Virgala, T. Lipták, L. Miková, F. Filakovský, and V. Bulej, "A novel approach for a inverse kinematics solution of a redundant manipulator," *Applied Sciences*, vol. 8, no. 11, 2018. [Online]. Available: https://doi.org/10.3390/app8112229

[19] N. Andrei, *Sequential Quadratic Programming*. Cham: Springer International Publishing, 2022, pp. 521–567. [Online]. Available: https://doi.org/10.1007/978-3-031-08720-2_15

[20] Y. Bai and S.-J. Hsieh, "A hybrid method using fabrik and custom ann in solving inverse kinematic for generic serial robot manipulator," *The International Journal of Advanced Manufacturing Technology*, vol. 130, pp. 4883–4904, 2024. [Online]. Available: https://doi.org/10.1007/s00170-023-12928-3

[21] G. Fang, Y. Tian, Z.-X. Yang, J. M. P. Geraedts, and C. C. L. Wang, "Efficient jacobian-based inverse kinematics with sim-to-real transfer of soft robots by learning," *IEEE/ASME Transactions on Mechatronics*, vol. 27, no. 6, pp. 5296–5306, 2022. [Online]. Available: https://doi.org/10.1109/TMECH.2022.3178303

[22] T. B. Moeslund and E. Granum, "A survey of computer vision-based human motion capture," *Computer Vision and Image Understanding*, vol. 81, no. 3, pp. 231–268, 2001. [Online]. Available: https://doi.org/10.1006/cviu.2000.0897

[23] A. Bhoi, "Monocular depth estimation: A survey," 2019. [Online]. Available: https://doi.org/10.48550/arXiv.1901.09402

[24] S. Saxena, A. Kar, M. Norouzi, and D. J. Fleet, "Monocular depth estimation using diffusion models," 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2302.14816

[25] M. S. Mahmoud and M. M. Hamdan, "Control of teleoperation systems in the presence of cyber attacks: A survey," *IAES International Journal of Robotics and Automation (IJRA)*, vol. 10, no. 3, pp. 235–260, 2021. [Online]. Available: https://doi.org/10.11591/ijra.v10i3.pp235-260

[26] Y. Gounine, P. K. Minaoui, and P. M. Elaroussi, "Cybersecurity in robotic telesurgery: Securing robot-console communications for safe remote surgery," in *2025 International Conference on Intelligent Systems: Theories and Applications (SITA)*, 2025, pp. 1–8. [Online]. Available: https://doi.org/10.1109/SITA67914.2025.11273620

[27] A. Bernadotte, "Cyber security for surgical remote intelligent robotic systems," in *2023 9th International Conference on Automation, Robotics and Applications (ICARA)*, 2023, pp. 65–69. [Online]. Available: https://doi.org/10.1109/ICARA56516.2023.10126050

[28] S. Iqbal, S. Farooq, K. Shahzad, A. W. Malik, M. M. Hamayun, and O. Hasan, "Securesurginet: A framework for ensuring security in telesurgery," *International Journal of Distributed Sensor Networks*, vol. 15, no. 9, p. 1550147719873811, 2019. [Online]. Available: https://doi.org/10.1177/1550147719873811

[29] M. Sabouri, "Cybersecurity of teleoperated quadruped robots: A systematic survey of vulnerabilities, threats, and open defense gaps," 2026. [Online]. Available: https://arxiv.org/abs/2602.23404

[30] H. Takanashi, A. Kosugi, K. Teranishi, T. Mizuya, K. Abe, and K. Kogiso, "Cyber-secure teleoperation with encrypted four-channel bilateral control," *IEEE Transactions on Control Systems Technology*, vol. 33, no. 6, pp. 2163–2177, 2025. [Online]. Available: https://doi.org/10.1109/TCST.2025.3577536

[31] D.-F. Hriţcan, A. Graur, D.-G. Bălan, and E.-M. Timofte, "A performance analysis of vpn technologies used in an iot environment," *Advances in Electrical and Computer Engineering*, vol. 25, no. 2, 2025. [Online]. Available: https://www.researchgate.net/profile/Edi-Timofte/publication/393410400_A_Performance_Analysis_of_VPN_Technologies_Used_in_an_IoT_Environment/links/686beba292697d42903d9e91/A-Performance-Analysis-of-VPN-Technologies-Used-in-an-IoT-Environment.pdf

[32] H. Jumakhan and A. Mirzaeinia, "Wireguard: An efficient solution for securing iot device connectivity: Regular research paper (csci-rtmc)," in *2023 International Conference on Computational Science and Computational Intelligence (CSCI)*, 2023, pp. 934–940. [Online]. Available: https://doi.org/10.1109/CSCI62032.2023.00156

[33] C. M. Purba and P. D. Ibnugraha, "Securing data transmission in iot devices using wireguard vpn," in *2025 IEEE International Conference on Artificial Intelligence and Mechatronics Systems (AIMS)*, 2025, pp. 1–5. [Online]. Available: https://doi.org/10.1109/AIMS66189.2025.11229500

[34] T. T. Nguyen, C. Wilson, I. Khan, and J. Dalins, "Object detection approaches to identifying hand images with high forensic values," *2024 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 2024. [Online]. Available: https://doi.org/10.1109/SMC54092.2024.10831640

[35] B. Kristianto, C. Dewi, H. D. Purnomo, K. D. Hartomo, and S. Z. M. Hashim, "Utilizing the yolov8 model for accurate hand recognition with complex background," *PeerJ Computer Science*, 2025. [Online]. Available: https://doi.org/10.7717/peerj-cs.3244

[36] J. Docekal, J. Rozlivek, J. Matas, and M. Hoffmann, "Human keypoint detection for close proximity human-robot interaction," in *2022 IEEE-RAS 21st International Conference on Humanoid Robots (Humanoids)*, 2022, pp. 450–457. [Online]. Available: https://doi.org/10.1109/Humanoids53995.2022.10000133

[37] L. A. O. Moreno and J. H. A. Alcántara, "An adaptation of fabrik algorithm for serial robot's inverse kinematics," in *2022 IEEE International Conference on Engineering Veracruz (ICEV)*, 2022, pp. 1–6. [Online]. Available: https://doi.org/10.1109/ICEV56253.2022.9959663

[38] A. Aristidou and J. Lasenby, "Fabrik: A fast, iterative solver for the inverse kinematics problem," *Graphical Models*, vol. 73, no. 5, pp. 243–260, 2011. [Online]. Available: https://doi.org/10.1016/j.gmod.2011.05.003

[39] *ESP32-D0WDQ6 Datasheet*, Espressif, July 2026, rev 5.3. [Online]. Available: https://documentation.espressif.com/esp32_datasheet_en.pdf

[40] I. E. T. Force, "Rfc 6347: Datagram transport layer security version 1.2," 2012. [Online]. Available: https://www.rfc-editor.org/info/rfc6347/

[41] *ATmega328P Datasheet*, Atmel Corporation, November 2016, rev D. [Online]. Available: https://docs.arduino.cc/resources/datasheets/Atmel-42735-8-bit-AVR-Microcontroller-ATmega328-328P_Datasheet.pdf

[42] Arduino, "Braccio." [Online]. Available: https://github.com/arduino-libraries/Braccio

[43] G. Casiez, N. Roussel, and D. Vogel, "1 € filter: a simple speed-based low-pass filter for noisy input in interactive systems," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, ser. CHI '12. New York, NY, USA: Association for Computing Machinery, 2012, p. 2527–2530. [Online]. Available: https://doi.org/10.1145/2207676.2208639

[44] "Springrc sr311," ServoDatabase.com, September 2026. [Online]. Available: https://servodatabase.com/servo/springrc/sr311

[45] "Springrc sr431," ServoDatabase.com, September 2026. [Online]. Available: https://servodatabase.com/servo/springrc/sr431

[46] WonderNetwork, "Global ping statistics." [Online]. Available: https://wondernetwork.com/pings

[47] P. Beeson and B. Ames, "Trac-ik: An open-source library for improved solving of generic inverse kinematics," 11 2015. [Online]. Available: https://doi.org/10.1109/HUMANOIDS.2015.7363472

Appendix A

GA Appendix

GA	Requirement	Justification and section in the report
1	Problem-solving	In this project I needed to implement intuitive spatial mapping to enable teleoperation. The relevant literature was reviewed in section 2.3 to determine the best approach. I then designed a software implementation around a particular methodology in chapter 4, extending it to fully solve the problem.
4	Investigations, experiments and data analysis	I designed experiments to benchmark the various subsystems, such as in section 5.3. The raw data was organised and process, giving the results shown in chapter 7. I then discussed these results in the context of the literature and used it to inform conclusions, seen in chapter 8.
5	Use of engineering tools	To implement the final solution, I used various programming languages and their libraries (see any subsystem), as well the PlatformIO IDE to flash and debug the firmware (see section 5.3). A voltmeter and oscilloscope were used to debug the serial communication. Wireshark was used to confirm that packets were encrypted.
6	Professional and technical communication (Long report)	This report is written in an academic format, and uses engineering documentation such as pseudocode (see Algorithm 1) to communicate complex ideas.
8	Individual work	This project is my own work, except where I have stated otherwise. The project incorporates aspects from different disciplines; computer vision (chapter 3), embedded systems, and cybersecurity (chapter 5).
9	Independent learning ability	This project has required knowledge about technologies that were not directly (through lectures) taught in the curriculum, such as inverse kinematics (see chapter 4).

Appendix B

Ethics Approval Letter



PRE-SCREENING QUESTIONNAIRE OUTCOME LETTER

STU-EBE-2026-PSQ002682

2026/08/14

Dear Neo Vorsatz,

Your Ethics pre-screening questionnaire (PSQ) has been evaluated by your departmental ethics representative. Based on the information supplied in your PSQ, it has been determined that you do not need to make a full ethics application for the research project in question.

You may proceed with your research project titled:

Secure Teleoperation of a Robot Arm using Computer Vision

Please note that should aspect(s) of your current project change, you should submit a new PSQ in order to determine whether the changed aspects increase the ethical risks of your project. It may be the case that project changes could require a full ethics application and review process.

Regards,

Faculty Research Ethics Committee

Appendix C

Inverse Kinematics Pseudocode

Algorithm 2 Move Joint under Forward Reference Constraints

Require: An array J of joints, and the index j of the joint being moved,
Ensure: Joint $J[j]$ is positioned & oriented under the forward reference constraints of $J[j+1]$

1: **function** ForwardReference(J, j)
    // Shorthand for "previous joint"
2: $P \leftarrow J[j+1]$

    // Update joint orientation and position
3: $J[j].\vec{axis}_{link} \leftarrow P.\vec{axis}_{link}$
4: $J[j].\vec{axis}_{aux} \leftarrow P.\vec{axis}_{aux}$
5: $J[j].\vec{pos} \leftarrow P.\vec{pos} - J[j].\vec{axis}_{link} J[j].len_{link}$

6: **end function**

Algorithm 3 Move Joint under Forward Hinge Constraints

Require: An array J of joints, and the index j of the joint being moved
Ensure: Joint $J[j]$ is positioned & oriented under the forward hinge constraints of $J[j+1]$

function ForwardHinge(J, j)
    // Shorthand for "previous joint"
    $P \leftarrow J[j+1]$

    // Project distance $\vec{d}$ onto the plane perpendicular to $P.\vec{axis}_{aux}$
    $\vec{d} \leftarrow P.\vec{pos} - J[j].\vec{pos}$
    $\vec{l} \leftarrow \vec{d} - (\vec{d} \cdot P.\vec{axis}_{aux})P.\vec{axis}_{aux}$

    // Update joint orientation
    $mag_l \leftarrow |\vec{l}|$
    if $mag_l = 0$ **then**
        $J[j].\vec{axis}_{link} \leftarrow P.\vec{axis}_{link}$
    else
        $J[j].\vec{axis}_{link} \leftarrow (\vec{l}/mag_l)$
    end if
    $J[j].\vec{axis}_{aux} \leftarrow P.\vec{axis}_{aux}$

    // Check joint limits
    $sign \leftarrow \text{sgn}((J[j].\vec{axis}_{link} \times P.\vec{axis}_{link}) \cdot J[j].\vec{axis}_{aux})$
    $P.value \leftarrow \cos^{-1}(J[j].\vec{axis}_{link} \cdot P.\vec{axis}_{link})sign$
    if P is limited **then**
        if $P.value > P.max$ **then**
            $P.value \leftarrow P.max$
            $J[j].\vec{axis}_{link} \leftarrow P.\vec{axis}_{link}\cos(P.value)$
            $J[j].\vec{axis}_{link} \leftarrow J[j].\vec{axis}_{link} + (P.\vec{axis}_{link} \times P.\vec{axis}_{aux})\sin(P.value)$
        else if $P.value < P.min$ **then**
            $P.value \leftarrow P.min$
            $J[j].\vec{axis}_{link} \leftarrow P.\vec{axis}_{link}\cos(P.value)$
            $J[j].\vec{axis}_{link} \leftarrow J[j].\vec{axis}_{link} + (P.\vec{axis}_{link} \times P.\vec{axis}_{aux})\sin(P.value)$
        end if
    end if

    // Update joint position
    $J[j].\vec{pos} \leftarrow P.\vec{pos} - J[j].\vec{axis}_{link}J[j].len_{link}$
end function

Algorithm 4 Move Joint under Forward Pivot Constraints

Require: An array J of joints, and the index j of the joint being moved
Ensure: Joint $J[j]$ is positioned & oriented under the forward pivot constraints of $J[j+1]$

function ForwardPivot(J, j)
    // Shorthand for "previous joint"
    $P \leftarrow J[j+1]$

    // Update joint position
    $J[j].\vec{axis}_{link} \leftarrow P.\vec{axis}_{link}$
    $J[j].\vec{pos} \leftarrow P.\vec{pos} - J[j].\vec{axis}_{link} J[j].len_{link}$

    // Determine auxiliary axis direction
    $\vec{d} \leftarrow J[j].\vec{pos} - J[j-1].\vec{pos}$
    $\vec{a} \leftarrow J[j].\vec{axis}_{link} \times \vec{d}$
    $mag_a \leftarrow |\vec{a}|$
    if $mag_a = 0$ **then**
        $\vec{a} \leftarrow P.\vec{axis}_{aux}$
    else
        $\vec{a} \leftarrow \vec{a}/mag_a$
    end if

    // Correct the direction of the auxiliary axis
    $J[j].\vec{axis}_{aux} \leftarrow \vec{a}\,\mathrm{sgn}(J[j].\vec{axis}_{aux} \cdot \vec{a})$

    // Check joint limits
    $sign \leftarrow \mathrm{sgn}((J[j].\vec{axis}_{aux} \times P.\vec{axis}_{aux}) \cdot J[j].\vec{axis}_{link})$
    $P.value \leftarrow \cos^{-1}(J[j].\vec{axis}_{aux} \cdot P.\vec{axis}_{aux})sign$
    if P is limited **then**
        if $P.value > P.max$ **then**
            $P.value \leftarrow P.max$
            $J[j].\vec{axis}_{aux} \leftarrow P.\vec{axis}_{aux}\cos(P.value)$
            $J[j].\vec{axis}_{aux} \leftarrow J[j].\vec{axis}_{aux} - (P.\vec{axis}_{link} \times P.\vec{axis}_{aux})\sin(P.value)$
        else if $P.value < P.min$ **then**
            $P.value \leftarrow P.min$
            $J[j].\vec{axis}_{aux} \leftarrow P.\vec{axis}_{aux}\cos(P.value)$
            $J[j].\vec{axis}_{aux} \leftarrow J[j].\vec{axis}_{aux} - (P.\vec{axis}_{link} \times P.\vec{axis}_{aux})\sin(P.value)$
        end if
    end if
end function

Algorithm 5 Move Joint under Forward Prismatic Constraints

Require: An array J of joints, and the index j of the joint being moved,
Ensure: Joint $J[j]$ is positioned & oriented under the forward prismatic constraints of $J[j+1]$

1: **function** ForwardPrismatic(J, j)
    // Shorthand for "previous joint"
2: $P \leftarrow J[j+1]$

    // Update joint orientation
3: $J[j].\vec{axis}_{link} \leftarrow P.\vec{axis}_{link}$
4: $J[j].\vec{axis}_{aux} \leftarrow P.\vec{axis}_{aux}$

    // Calculate joint length
5: $\vec{d} \leftarrow P.\vec{pos} - J[j].\vec{pos}$
6: $P.value \leftarrow (\vec{d} \cdot J[j].\vec{axis}_{link}) - J[j].len_{link}$

    // Check joint limits
7: **if** P is limited **then**
8: **if** $P.value > P.max$ **then**
9: $P.value \leftarrow P.max$
10: **else if** $P.value < P.min$ **then**
11: $P.value \leftarrow P.min$
12: **end if**
13: **end if**

    // Update joint position
14: $J[j].\vec{pos} \leftarrow P.\vec{pos} - J[j].\vec{axis}_{link}(P.value + J[j].len_{link})$
15: **end function**

Appendix D

Example IK Acceptance Test Scenarios

Table D.1: Example Inverse Kinematics Acceptance Test Scenarios for RQ1

Setting	**ATP1.1**	**ATP1.2**	**ATP1.3**
Base - `pos`	1mm, 2mm, 3mm	-2mm, 0, 1mm	-1mm, 2mm, -2mm
Base - `axis_link`	0, 0, 1mm	0, 0, 1mm	0, 0, 1mm
Base - `axis_aux`	1mm, 0, 0	1mm, 0, 0	1mm, 0, 0
Joint - `joint_type`	Hinge	Pivot	Prismatic
Joint - `len_link`	5mm	5mm	1mm
Joint - `limited`	false	false	false
Joint - `min`	N/A	N/A	N/A
Joint - `max`	N/A	N/A	N/A
Target - `pos`	1mm, 7mm, 3mm	-2mm, 0, 6mm	-1mm, 2mm, 5mm
Target - `axis_link`	0, 1mm, 0	0, 0, 1mm	0, 0, 1mm
Target - `axis_aux`	1mm, 0, 0	0, 1mm, 0	1mm, 0, 0
expected `value`	-90°	90°	6mm

Table D.2: Example Inverse Kinematics Acceptance Test Scenarios for RQ2

Setting	**ATP2.1**	**ATP2.2**	**ATP2.3**
Base - `pos`	1mm, 2mm, 3mm	-2mm, 0, 1mm	-1mm, 2mm, -2mm
Base - `axis_link`	0, 0, 1mm	0, 0, 1mm	0, 0, 1mm
Base - `axis_aux`	1mm, 0, 0	1mm, 0, 0	1mm, 0, 0
Joint - `joint_type`	Hinge	Pivot	Prismatic
Joint - `len_link`	4mm	7mm	2mm
Joint - `limited`	false	false	false
Joint - `min`	N/A	N/A	N/A
Joint - `max`	N/A	N/A	N/A
Target - `pos`	3mm, -0.83mm, 5.83mm	-1mm, 0, 4mm	-3mm, 2mm, 1mm
Target - `axis_link`	1mm, 0, 0	0, 0, 1mm	-1mm, 0, 0
Target - `axis_aux`	0, -1mm, 0	0, -1mm, 0	0, 1mm, 0
expected `value`	45°	-90°	1mm

Table D.3: Example Inverse Kinematics Acceptance Test Scenarios for RQ3

Setting	**ATP3.1**	**ATP3.2**	**ATP3.3**
Base - `pos`	1mm, 2mm, 3mm	-2mm, 0, 1mm	-1mm, 2mm, -2mm
Base - `axis_link`	0, 0, 1mm	0, 0, 1mm	0, 0, 1mm
Base - `axis_aux`	1mm, 0, 0	1mm, 0, 0	1mm, 0, 0
Joint - `joint_type`	Hinge	Pivot	Prismatic
Joint - `len_link`	5mm	5mm	1mm
Joint - `limited`	true	true	true
Joint - `min`	-30°	-45°	-1mm
Joint - `max`	45°	60°	4.5mm
Target - `pos`	1mm, 7mm, 3mm	-2mm, 0, 6mm	-1mm, 2mm, 5mm
Target - `axis_link`	0, 1mm, 0	0, 0, 1mm	0, 0, 1mm
Target - `axis_aux`	1mm, 0, 0	0, 1mm, 0	1mm, 0, 0
expected `value`	-30°	60°	4.5mm

Appendix E

AI Usage

My use of artificial intelligence in writing this report has been limited to:

E.1 Inline Code Suggestions

I used a few inline code suggestions generated by Mistral's Codestral inside Visual Studio Code. An example of what it suggests can be seen in Figure E.1.

```
% Appendix E ================================
\chapter{AI Usage}\label{app:AI_usage}

My use of artificial intelligence in writing this report has been
limited to:

\section{Inline Code Suggestions}
Inline code suggestions, generated by Mistral's Codestral inside
Visual Studio Code. An example of what it suggests can be seen in
\autoref{fig:ai_code_suggestion
```

Figure E.1: Screenshot of VS Code, showing inline code suggestion

E.2 Google Gemini's Advice on LaTeX syntax

Very often, I had to look up LaTeX syntax. I use the Google search engine, so Gemini's results often pop up, and it's faster than opening the documentation Overleaf provides. An example can be seen in Figure E.2.

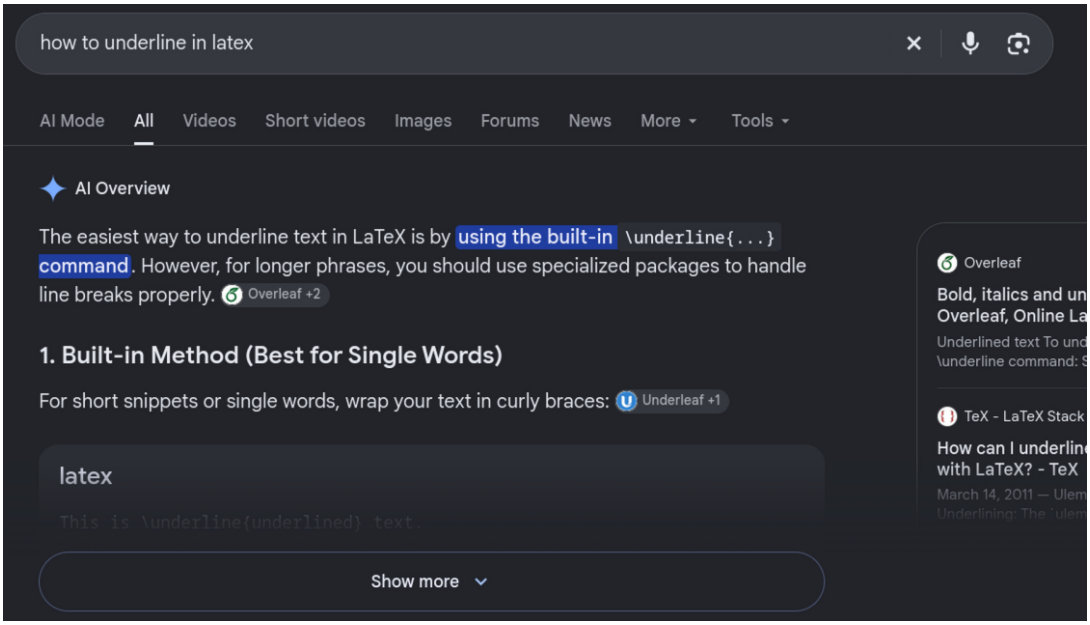


Figure E.2: Screenshot of Firefox showing Google's search results